Document number: P0889R1
Project: Programming Language C++
Audience: Evolution Working group, Core Working group

Antony Polukhin, Yandex.Taxi Ltd, <antoshkka@gmail.com>, <antoshkka@yandex-team.ru>

Date: 2019-01-09

Ultimate copy elision

> "...take no action on CWG #6 issue until an interested party produces a paper with analysis and a proposal."
>
> *— CWG #6*

Significant changes since P0889R0 are marked with blue.

☐ Show deleted lines from P0889R0.

I. Quick Introduction

[class.copy.elision] in the current working draft [N4713] provides a whitelist of cases when copy elision is permitted. Those rules are good! However they do not take into account that modern compilers could inline functions and do other optimizations.

This paper motivates and proposes to relax the [class.copy.elision] rules in order to allow compilers to produce much better code by mixing copy elision, inlining and other optimizations. It also addresses CWG #6, CWG #1049, and CWG #1579. It also gives a non-guaranteed solutions for CWG #2327.

II. The pitfall

A. Teaching practice

For decades, almost all teaching materials were telling people to decompose their programs into functions for maintainability and code re-usability. That good advice leads to code with numerous functions. Compiler developers noted that and started inlining functions more aggressively.

Currently inlining is one of the major optimizations [Understanding Compiler Optimization, 26m20s by Chandler Carruth] and compilers inline a lot.

B. Copy elision rules

Current rules for copy elision mostly assume that a function from source code remains a function in a binary. This works perfectly fine in a world without inlining, aliasing reasoning, and link time optimization. But if the function is inlined, then the compiler "sees" the whole function body along with function parameters. This unleashes a whole new world of possible optimizations (See section III for examples), but existing copy elision rules prevent those optimizations.

Our current rules are suboptimal for modern compilers: they prevent optimizations.

C. std::move/rvalues is not a panacea

We have `std::array`, `std::basic_string`, `std::function`, `std::variant`, `std::optional` and other classes that may store a lot of data on the stack. "Moving" instances of those classes may result in copying a lot of bytes.

Copy elision could be more profitable.

D. std::move/rvalues rules may be obscure

What is the optimal way to return something from a function? For named object we must just return. For a subobject we must return it with `std::move`. For a function parameter we must return it by `std::move`. If we return a reference to a local object, we have to `std::move` it. We must also return elements from structured binding via `std::move`, but only if the element is not a reference to an value returned by reference before decomposition... Do we have to apply `std::move` for a member function call on a local variable when the function call returns reference?..

Beginners and even some advanced programmers do not always know about those obscure rules and already assume that the compiler will do the job for them.

E. In sum

We have for decades been teaching people to write functions. WG21 was improving the C++ language by providing features for advanced programmers to optimize the functions, leaving behind the abilities of modern compilers to do that job sometimes better and sometimes out-of-the-box.

III. History of the problem

First copy elision rules were proposed in 1995 in N0641 "Copy optimization". Those rules were quite close to what was proposed in the early versions of this paper. Here they are:

```
Whenever a class object is copied and the implementation can prove that either the original or the copy
will never again be used, an implementation is permitted to treat the original and the copy as two
different ways of referring to the same object and not generate a copy at all. In that case, the object
is destroyed at the later of the times when the original and the copy would have been destroyed without
the optimization.
```

N0641 "Copy optimization" was adopted into the C++ WD in N0661 "WG21 Meeting No. 12". Two years later issues were found and described in N1079 "Core WG List of new issues":

> I think the problem can be summarized by saying that objects can bind resources, and even if an object is not used, the resource it binds might be. The kind of thing that might happen is:
>
> ```
> Thing x = /* some value */;
> SubThing y = x.extract_portion();
> Thing z = x;
> z.clobber_portion();
> // now try to fetch the value of y
> ```
>
> If x is never used again, the compiler is entitled to alias z and x. However, if y actually refers to part of the storage that x used, clobbering z (which is an alias to x) might also clobber y.

That issue was discussed multiple times, including N1108 "WG21 Meeting No. 19". In that paper two cases where Core especially wanted to allow copy elisions were highlighted:

- elision of copy of temporary values,
- elision of copy of return value.

> Core would conduct further work to look at other optimizations.

Finally, in N1182 "Proposed Resolutions for Core Language Issues 6, 14, 20, 40, and 89" appeared the copy elision rules close to the ones we have now. Since then we have an open issue CWG #6 as a reminder that C++ could do better than now.

IV. Analysis

Part I

As was noted in N1079 object could "bind" its resources to other objects:

```
char* some;

struct binds_resource {
    binds_resource() = default;
    binds_resource(const binds_resource& other) = default;

    void bind() {
        some = data.data();
    }

    string data = "A";
};


void test() {
    binds_resource original{};
    original.bind();

    shared_state copy{original};
    copy.data = "B";
    assert(some != copy.data);
}
```

If in the above example we enable to elide `copy` and use `original` instead, then the assertion will **fail**.

Let's concentrate on a simpler case, when the original object is not accessed between construction and copying and when the original object is not accessed after copying and destruction. For such a simple case we still may get into trouble, if the original object "binds" its resources in constructor:

```
char* some;

struct binds_resource {
    binds_resource()
        : data{"A"}
    {
        bind();
    }

    binds_resource(const binds_resource& other) = default;

    void bind() {
        some = data.data();
    }

    string data = "A";
};


void test() {
    binds_resource original{};
    shared_state copy{original};
    copy.data = "B";
    assert(some != copy.data);
}
```

Such binding could also happen via constructor output parameters:

```
struct binds_resource {
    binds_resource(char** out)
        : data{"A"}
    {
        *out = data.data();
    }

    binds_resource(const binds_resource& other) = default;
    string data = "A";
};


void test() {
    char* some;
    binds_resource original{&some};

    shared_state copy{original};
    copy.data = "B";
    assert(some != copy.data);
}
```

More problems. We may get into troubles with eliding copies of resources that are used by different threads:

```
mutex m;

void takes_by_copy(std::string v) noexcept {
    m.unlock();
    v += "Oops."; // must work with copy!
}

void test() {
    m.lock();

    std::string original{"some string that is too big for SSO"};

    std::thread t{
        [&original]() {
            m.lock();
            // under the lock
            // ...
        }
    };

    takes_by_copy(original);

    t.join();
}
```

Note that the scope of the Source should remain the same or extend after the elision:

```
struct locked_mutex {
    mutex m{};
    lock_guard<mutex> lock{m};
};

void test() {
    shared_ptr<locked_mutex> original = std::make_shared<mutex>();

    // acessing resource that should be protected by lock

    shared_state copy{original};
}
```

Finally, we may get into troubles with eliding copies between different threads of execution:

```
void test() {
    some_shared_spin_guard original{spinlock};

    std::thread{
        [](const auto& lock) { /* under the lock */ },
        original
    }.detach();

    // original should be destroyed here and should release the lock here!
}
```

Conclusion: carefully investigating all the above cases we could enable the copy elisions for the case when:

- Source is not accessed between construction and copy
- and Source is not accessed between copy and destruction
- and Source and Target belong to the same thread of execution
- and Source and Target have automatic storage duration
- and Source construction does not access any global variables and does not return values through output parameters
- and scope of Source after the elision extends to the scope of the target

Benefits: With above rules we get copy elisions in the following popular cases:

Case 1. CWG #6

```
void takes_by_copy(std::string v) noexcept { /* ... */ }

void example() {
    string v{"some string that is too big for SSO"};
    takes_by_copy(v);
}
```

Case 2. CWG #6

```
void takes_by_creference(const std::string& v) noexcept {
    std::string copy{v};
    /* ... */
}

void example() {
    return takes_by_creference("some string that is too big for SSO");
}
```

As Jens Maurer noted a sligtly modified example leads to UB: 'Given your copy elision rules, it seems "copy" could alias "s", introducing undefined behavior (see 9.1.7.1 [dcl.type.cv] p4).'

```
void takes_by_creference(const std::string& v) noexcept {
    std::string copy{v};
    copy += "blah";
}

void example() {
    const std::string s("some string);
    return takes_by_creference(s);
}
```

This paper proposes to adjust [dcl.type.cv] p4 making that behavior well defined.

Case 3. CWG #1049

```
struct B {
   string a;
   B(const string& a): a(a) { }
};

int main() {
   B("some string that is too big for SSO");
}
```

Part II: Source leaves scope

Now let's concentrate on another case: when the source object is destroyed right after the copy/move construction:

```
char* some;

struct binds_resource {
    binds_resource()
        : data{"A"}
    {
        bind();
    }

    binds_resource(const binds_resource& other) = default;

    void bind() {
        some = data.data();
    }

    string data = "A";
};

void test() {
    auto generate = []() {
        binds_resource source{};
        return source; // NRVO is possible
    };

    auto copy = generate();

    copy.data = "B";
    assert(some != copy.data); // UB
}
```

Existing rules already rely on the fact that the object that leaves scope should not be referenced outside the scope. Let's change example to have no UB:

```
shared_ptr<int> some;

struct shares_state {
    shares_state()
```

```
        : data{make_shared<int>(42)}
    {
        bind();
    }

    shares_state(const shares_state& other) {
        data = make_shared<int>(*other.data);
    }

    void bind() {
        some = data;
    }

    shared_ptr data;
};

void test() {
    auto generate = []() {
        shares_state source{};
        return source; // NRVO is possible
    };

    auto copy = generate();

    *copy.data = 0;
    assert(*some != *copy.data); // Implementation defined
}
```

In other words: **we don't have to check for "binding" if the scope of Source ends right after the copy/move construction**.

Conclusion: we can relax the rule in "Part I" for the cases when the scope of Source ends immediately after the copy/move construction:

- Source is not accessed between copy and destruction
- and Source and Target have automatic storage duration
- and Source and Target belong to the same thread of execution
- and scope of Source after the elision extends to the scope of the target
- and either
  - scope of Source ends after the copy/move construction of Target
  - or Source is not accessed between construction and copy, and Source construction does not access any global variables and does not return values through output parameters

Benefits: With above rules we get copy elisions in the following popular cases:

Case 1. Returning a function parameter

```
string callee(string v) {
    // any code
    return v;  // copy could be elided
}

auto example() {
    auto v = callee("some string that is too big for SSO");
    return v.size();
}
```

Case 2. Returning a reference to a local

```
string callee() {
    string v{"some string that is too big for SSO"};
    const auto& ref = v;
    return ref;  // copy could be elided
}

auto example() {
    auto v = callee();
    return v.size();
}
```

Case 3. Move by mistake

```
string callee() {
    string v{"some string that is too big for SSO"};
    return std::move(v);  // copy could be elided
}

auto example() {
    auto v = callee();
    return v.size();
}
```

Part III: Simple destructors and constructors

It may be hard to implement the above copy elision logic without taking additional care of compiler specifics. Some compilers at certain stages of optimization may inline constructors/destructors and just call the constructors/destructors of the members. So in the assembly (and probably in the IR) code like:

```
void test1() {
    std::pair<A, A> pair;
    (void)pair;
}
```

may look like:

```
test1():
  push rbp
  sub rsp, 16
  mov rdi, rsp
  call A::A()
  lea rdi, [rsp+8]
  call A::A()
  lea rdi, [rsp+8]
  call A::~A()
  mov rdi, rsp
  call A::~A()
  add rsp, 16
  pop rbp
  ret
```

With such representations it could be hard to distinguish `pair` objects from `A` objects. To minimize the changes required to implement the elision rules this paper proposes to allow eliding subobjects, while the elision rules from "Part I" and "Part II" are not violated.

Note that the proposed changes may change the destruction order of nested objects:

```
string callee() {
    pair<string, string> v = foo();
    return v.second;
}

auto test() {
    auto v = callee();
    return v.size();
}


auto pseudocode_with_proposed_changes() {
    pair<string, string> v = foo();
    v.first.~string();
    return v.size();
    // do not call destructor of `v`, just destroy `v.second`
}
```

Let's invent an example where such optimization could break some code:

```
struct local_vector {
    pmr::monotonic_buffer_resource mr;
    pmr::vector<int> data{&mr};
};

auto callee() {
    local_vector lv;
    lv.data.resize(1000, 42);

    return lv.data; // copying
}

auto test() {
    auto v = callee(); // `mr` is destroyed, but during copying default memory resource is used
    /*...*/
}
```

With the proposed copy elision rules the above example becomes broken: there'll be no copying so the old memory resource will be used, that is destroyed.

Note that the example is already shaky. Minor changes result in broken code:

```
auto callee0() {
    monotonic_buffer_resource mr;
    pmr::vector<int> v(&mr);
    v.resize(1000, 42);
    return v;
}

auto test0() {
    auto v = callee(); // Already broken
}
```

In this paper we'd like to suggest enabling copy elision for the above rules and make an emergency hatch for disabling copy/move elisions for cases like above.

Conclusion: relax the rule in "Part II" by allowing copy elision for subobjects and make an "emergency hatch" for cases when copies are required. See "VII. Proposed wording" for the wording.

Benefits: With above rules we get copy elisions in the following popular cases:

Case 1. Returning a subobject

```
static string callee() {
    pair<string, string> v;
    return v.second;
}
```

Case 2. Returning an element from a structured binding

```
static string callee() {
    auto [f, s] = pair{};
    return s;
}
```

Case 3. CWG #1579

```
  optional<T> foo() {
    T t;
    ...
    // inlining the constructor of `optional` may result in internal storage of optional being
    // initialized by constructing T from `t`, which scope ends.
    return t;
  }
```

V. Implementability example.

Let's take a look at some pseodocode representing the C++ program after the inlining optimization:

```
struct A {
    A() __attribute__ ((pure)); // "pure" is detected by the compiler
    A(const A&);
    A(A&&) = default __attribute__ ((pure)); // "pure" is detected by the compiler
    ~A();

    int some_function();
private:
    // members
};

int caller() {
    A a;
    a.some_function();
    A b{a};             // scope of `a` ends immediately after the copy constructor call, eliding `b` and using `a` with extended
    [[end of scope `a`]]

    A c{b}; // `a` is accessed between this line, no copy elision allowed
    A d{c}; // `c` is not accessed after copy construction and its copy constructor is a "pure" function. OK to elide
    return d.some_function()
}
```

VI. Addressing common concerns

Concern 1: Eliding copy/move constructors could break user code

Response: For the rules from "Part I" nothing should get broken. For the remaining cases... Yes, but only if the following generally-accepted constraint for a copy constructor is not satisfied: "After the definition `T u = v;`, `u` is equal to `v`".

Although the constraint seems very restrictive at first, that constraint is satisfied by every sane copy constructor. Moreover the C++ Language and Standard Library heavily rely on it:

- The Ranges TS requires that objects after copy/assignment/move are equal (N4651: "After the definition T u = v;, u is equal to v" in concept CopyConstructible. The same rules can be found in concept Assignable and concept Swappable).
- The standard library implicitly requires objects after copy/assignment/move to be equal in [container.requirements.general].
- Algorithms do not work well if that constraint is not satisfied, as some of the complexities in [alg*] are described as "At most .* swaps" or "Approximately .* swaps". Moreover, algorithms sometimes do not specify the order of copying/swapping so if the requirement is not satisfied the results could be different on different platforms.
- [class.copy.elision] implicitly relies on that constraint.
- WG21 implicitly relied on that constraint in "P0135R1: Wording for guaranteed copy elision through simplified value categories".

In other words, WG21 has been relying on that constraint for a long time and classes that violate that constraint are already unportable.

Concern 2: The examples do not look like a real world code. Nobody writes such bad code

Response: Examples are simplified just to show that the optimization is possible.

Real code would have functions located in different headers, more code will be in the function body. We searched the Yandex code base for `return.*first;` and `return.*second;` and found thousands of matches. Note that we searched only for a single optimization case for only a `std::pair`. Tuples, aggregates and more complex data types are also affected.

Concern 3: Advanced optimizations could affect compilation time

That depends. Making a high level optimization could be faster than doing a lot of low level optimizations. For example removing the copy constructor and destructor could be faster than inlining both and optimizing all the internals.

Anyway, to implement or not to implement a particular optimization is up to the compiler developers. This proposal just attempts to untie their hands and allow more optimizations.

Concern 4: It is impossible to implement some of the optimizations right now.

Response: This proposal does not require any of the optimizations from examples. The proposal simply attempts to relax copy elision rules to allow those optimizations someday.

Concern 5: We want an emergency hatch to disable copy elision for particular places.

Response: Our usual practice to disable optimizations for a variable is to make it `volatile`. This feature must be kept.

Concern 6: The optimizations are not guaranteed, so users still have to write `std::move`

Response: That's true. Proposals for automatically applying `std::move` may be a good idea. Such proposals would be more restrictive than the copy elision rules because we have no control over all the compiler inlining and reasoning logic. We can not guarantee that all the compilers would be able to inline some function or would be able to understand that the reference references a local object.

Moving in both directions would produce better results:

- simplifying copy elision rules will remove border cases when `std::move` hurts performance
- implicit casting to rvalues will simplify the code and allow writing fewer `std::moves`
- implicit casting to rvalues and more generic copy elision rules will produce faster programs out-of-the-box. Users will have to worry less for performance. This will simplify the language usage and teaching materials (See "D. std::move/rvalues rules may be obscure").

Concern 7: How this would change the guidelines? How shall users write their code to get the benefits of this optimization?

Response: This optimization won't change the guidelines in the nearest future. Treat this optimization as one of the compiler optimizations that you could not directly control, like inlining, jump threading, loop fusion, or common subexpression elimination.

Some guidelines may change after major compilers adopt the optimization. In that case, there could be found a common pattern that triggers it and that pattern could be taught. Probably the existing guidelines some day would evolve into "If you've got a function bigger than 15 lines of code, you may use `std::move` for returning objects that are not constructed at the beginning of the function. Otherwise just return the value."

Concern 8: Extracting a subobject from an object is scary. I can no longer assume that the object I construct as a subobject is in any way part of myself, nor can it assume that any sibling subobjects will always be around.

Response: The proposed wording makes sure that the subobject is not used between copy/move construction and destruction. An object that is constructed as a subobject is a part of the object as long as you use it as a subobject. As soon as you copy/move and destroy it, you can not use it any more by existing C++ rules. That's the place where the optimization steps in, removing the copy/move+destruction and reusing the subobject.

Concern 9: The problem is not that big because compilers could inline the constructors and destructors and then optimize the resulting code

Response: Yes, they do that. But the resulting code is still suboptimal, because even for `std::string` compilers could not optimize away all the dead stores. Consider the following example:

```
#include <utility>
#include <string>

std::pair<std::string, std::string> produce();

static std::string first_non_empty_move() {
    auto v = produce();
    if (!v.first.empty()) {
        return std::move(v.first);
    }
    return std::move(v.second);
}

int example_1_move() {
    return first_non_empty_move().size();
}
```

Note that `std::move` is used and everything must be optimal. But it's not, because with this paper's proposed copy elision rules, the resulting code could still be much shorter and with fewer conditional jumps:

Without copy elision **https://godbolt.org/g/3xZvtw**	**With copy elision** **https://godbolt.org/g/RiTmPE**

`example_1_move():`
`  push  rbp`
`  push  rbx`
`  sub  rsp, 104`
`  lea  rdi, [rsp+32]`
`  mov  rbx, rsp`
`  call  produce[abi:cxx11]()`
	`example_1_optimized():`
	`  push  rbx`
	`  sub  rsp, 64`
	`  mov  rdi, rsp`
	`  call  produce[abi:cxx11]()`
	`  mov  rax, QWORD PTR [rsp+8]`
	`  test  rax, rax`

```
    mov     rax, QWORD PTR [rsp+40]
    test    rax, rax
    je      .L2
    lea     rdx, [rbx+16]
    lea     rcx, [rsp+48]
    mov     QWORD PTR [rsp], rdx
    mov     rdx, QWORD PTR [rsp+32]
    cmp     rdx, rcx
    je      .L13
    mov     QWORD PTR [rsp], rdx
    mov     rdx, QWORD PTR [rsp+48]
    mov     QWORD PTR [rsp+16], rdx
.L4:
    mov     QWORD PTR [rsp+8], rax
    lea     rax, [rsp+48]
    mov     rdi, QWORD PTR [rsp+64]
    mov     QWORD PTR [rsp+40], 0
    mov     BYTE PTR [rsp+48], 0
    mov     QWORD PTR [rsp+32], rax
    lea     rax, [rsp+80]
    cmp     rdi, rax
    je      .L6
    call    operator delete(void*)
    jmp     .L6
.L2:
    lea     rax, [rbx+16]
    lea     rdx, [rsp+80]
    mov     QWORD PTR [rsp], rax
    mov     rax, QWORD PTR [rsp+64]
    cmp     rax, rdx
    je      .L14
    mov     QWORD PTR [rsp], rax
    mov     rax, QWORD PTR [rsp+80]
    mov     QWORD PTR [rsp+16], rax
.L8:
    mov     rax, QWORD PTR [rsp+72]
    mov     QWORD PTR [rsp+8], rax
.L6:
    mov     rdi, QWORD PTR [rsp+32]
    lea     rax, [rsp+48]
    cmp     rdi, rax
    je      .L9
    call    operator delete(void*)
.L9:
    mov     rdi, QWORD PTR [rsp]
    add     rbx, 16
    mov     rbp, QWORD PTR [rsp+8]
    cmp     rdi, rbx
    je      .L1
    call    operator delete(void*)
.L1:
    add     rsp, 104
    mov     eax, ebp
    pop     rbx
    pop     rbp
    ret
.L14:
    movdqa  xmm0, XMMWORD PTR [rsp+80]
    movaps  XMMWORD PTR [rsp+16], xmm0
    jmp     .L8
.L13:
    movdqa  xmm0, XMMWORD PTR [rsp+48]
    movaps  XMMWORD PTR [rsp+16], xmm0
    jmp     .L4
```

```
    mov     ebx, eax
    jne     .L3
    mov     ebx, DWORD PTR [rsp+40]
.L3:
    mov     rdi, QWORD PTR [rsp+32]
    lea     rax, [rsp+48]
    cmp     rdi, rax
    je      .L4
    call    operator delete(void*)
.L4:
    mov     rdi, QWORD PTR [rsp]
    lea     rdx, [rsp+16]
    cmp     rdi, rdx
    je      .L1
    call    operator delete(void*)
.L1:
    add     rsp, 64
    mov     eax, ebx
    pop     rbx
    ret
```

Inlining the constructors and destructors and optimization of the resulting code fails in many cases:

- It is much harder to analyze the resulting code because it's bigger, touches more memory and registers.
- Constructors and destructors may be located in a separate translation unit, making that inlinement possible only during link time optimization.
- Constrcutor or destructor could be too big for inlinement and in that case no optimization is possible.

VII. Proposed wording: Ultimate solution for relaxing [class.copy.elision]

Adjust the [class.copy.elision] paragraph 1 to allow copy elision of all objects and subobjects:

> This elision of copy/move operations, called copy elision, is permitted in the following circumstances (which may be combined to eliminate multiple copies):
>
> – in a return statement in a function with a class return type, when the expression is the name of a non-volatile automatic object (other than a function parameter or a variable introduced by the exception-declaration of a handler (18.3)) with the same type (ignoring cv-qualification) as the function return type, the copy/move operation can be omitted by constructing the automatic object directly into the function call’s return object
>
> – in a throw-expression (8.17), when the operand is the name of a non-volatile automatic object (other than a function or catch-clause parameter) whose scope does not extend beyond the end of the innermost enclosing try-block (if there is one), the copy/move operation from the operand to the

exception object (18.1) can be omitted by constructing the automatic object directly into the exception object

– when the exception-declaration of an exception handler (Clause 18) declares an object of the same type (except for cv-qualification) as the exception object (18.1), the copy operation can be omitted by treating the exception-declaration as an alias for the exception object if the meaning of the program will be unchanged except for the execution of constructors and destructors for the object declared by the exception-declaration. [Note: There cannot be a move from the exception object because it is always an lvalue. — end note]

~~Copy elision is~~Above copy elisions are required where an expression is evaluated in a context requiring a constant expression (8.6) and in constant initialization (6.8.3.2). [Note: Copy elision might not be performed if the same expression is evaluated in another context. — end note]

Additionally, copy elision is allowed for any non-volatile object with automatic storage duration and its non-volatile subobjects if:

- the source is not accessed between copy/move and destruction, and
- the target has automatic storage duration, and
- the target belongs to the sources thread of execution, and
- the target is constructed by a copy or move constructor that accepts the source by non-volatile reference, and
- either
  - lifetime of the source ends after the copy/move construction of the target, or
  - the source is not accessed between construction and copy/move, and the source construction does not access any global variables and does not return values through output parameters.

For the above cases lifetime of the source after the elision extends to the lifetime of the target.

Adjust the [dcl.type.cv] paragraph 4 to avoid UB for elided copies:

Except that any class member declared mutable ([dcl.stc]) and any elided non-const target ([class.copy.elision]) can be modified, any attempt to modify ([expr.ass], [expr.post.incr], [expr.pre.incr]) a const object ([basic.type.qualifier]) during its lifetime ([basic.life]) results in undefined behavior.

VIII. Acknowledgements

Many thanks to Walter E. Brown for fixing numerous issues in draft versions of this paper.

Many thanks to Jens Maurer for providing multiple comments and for pointing to the UB.

Many thanks to Marc Glisse for providing a reference to CWG #1049.

Many thanks to Nicol Bolas for raising many concerns.